

# Sistema de Reserva de Canchas Deportivas

Microfrontends con Module Federation y microservicios Spring Boot

---

Pablo Jara Muñoz · Miguel Lara · Sebastián Uyaguari · Adrián Espinoza

29 de agosto de 2026

Maestría en Ingeniería de Software · Desarrollo de Aplicaciones Empresariales  
Universidad Politécnica Salesiana

# Lo que pedía el documento de alcance

Un club con canchas de **pádel, tenis y básquet**. Dos roles:

- **Socio:** consulta disponibilidad, reserva un bloque y cancela lo suyo.
- **Administrador:** gestiona el catálogo y los bloqueos, cancela cualquier reserva, activa e inactiva usuarios y mide el uso real.

Sobre eso, **ocho reglas de negocio (RN-01–RN-08)** y **seis criterios de aceptación (§7)**. Todo lo que sigue está atado a ese documento.

## Fuera de alcance (§3.5)

Pagos en línea · notificaciones · reservas recurrentes · torneos · app móvil nativa · BI avanzado.

No lo construimos, y es deliberado: está delimitado así en el propio documento.

# No fue *vibe coding*: fue especificación primero, con Spec Kit

/constitution → /specify → /plan → /tasks → /implement

Antes de generar la primera línea de código, los prompts de /specify dejaron escrito:

- **7 principios** de constitución, 2 de ellos no negociables.
- **48 requisitos funcionales** y **31 escenarios** *Given / When / Then*.
- **10 criterios de éxito medibles**, no adjetivos.
- **4 contratos OpenAPI** (20 rutas), acordados antes del primer *endpoint*.
- **134 tareas** derivadas del plan. Todas cerradas.

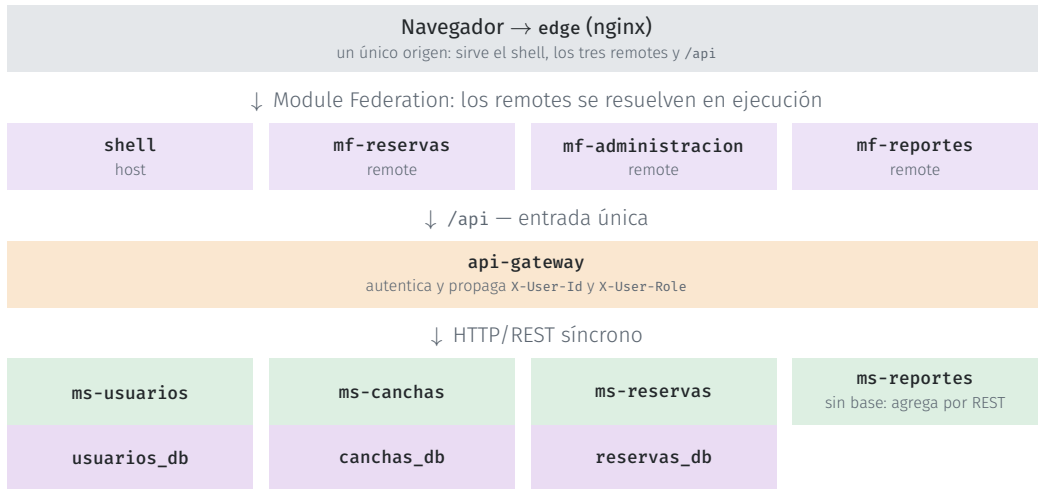
## Qué compró ese esfuerzo

Cada regla de negocio se puede seguir desde el §3.4 del alcance hasta una clase y una prueba con nombre propio.

El agente no inventó alcance: lo que no estaba especificado, no se construyó.

≈ 17 500 palabras de especificación antes de la primera línea de código.

# Once contenedores, un solo origen



Modelo C4 en Structurizr DSL. Las cuatro vistas completas, en el respaldo.

# Un host, tres remotes, desplegables por separado

```
pluginModuleFederation({
  name: 'shell',
  remotes: {
    mfReservas:      `mfReservas@${url.reservas}`,
    mfAdministracion: `mfAdministracion@${url.admin}`,
    mfReportes:      `mfReportes@${url.reportes}`,
  },
  shared: {
    react:           { singleton: true },
    'react-dom':     { singleton: true },
    'react-router-dom': { singleton: true },
  },
})
```

frontend/shell/rsbuild.config.ts

- Cada remote se carga **por URL, en ejecución**. El shell no los compila dentro.
- singleton: React y el enrutador se cargan una sola vez.
- El shell inyecta la sesión; el remote no la reimplementa.

## Desplegables por separado

Con el sistema en pie:

```
docker compose up -d
  --build mf-reportes
```

Se recarga el navegador y el cambio está. [El shell y los otros dos remotes nunca se reconstruyeron.](#)

# Cuatro servicios Spring Boot y un gateway

| Servicio    | Base        | Responsabilidad                                    |
|-------------|-------------|----------------------------------------------------|
| api-gateway | —           | Entrada única: autentica y propaga identidad y rol |
| ms-usuarios | usuarios_db | Registro, autenticación, usuarios y roles          |
| ms-canchas  | canchas_db  | Catálogo, horarios de atención y bloqueos          |
| ms-reservas | reservas_db | Disponibilidad, creación y cancelación             |
| ms-reportes | sin base    | Los cuatro indicadores, componiendo por REST       |

- **Contrato antes que código:** 4 documentos OpenAPI, 20 rutas, escritos antes del primer *endpoint*. Publicados en */swagger-ui*.
- Hexagonal: el dominio no conoce ni HTTP ni JPA.

**Quién decide quién eres:** el gateway valida la credencial contra `ms-usuarios` y escribe `X-User-Id` y `X-User-Role`. Los servicios confían en esas cabeceras *porque nadie más puede escribirlas*.

# Tres bases aisladas, y el aislamiento es verificable

- Tres bases, cada una con **su propio rol** de PostgreSQL: usuarios\_db, canchas\_db, reservas\_db.
- ms-reservas guarda el *identificador* de la cancha, no una clave foránea a otra base: la coherencia se valida llamando al servicio dueño del dato.
- Esquema y datos de demostración los crea Flyway al arrancar (E4): ningún paso manual.
- ms-reportes no tiene rol porque no tiene base.

El rol de ms-reservas intentando leer la base de canchas:

```
$ psql "postgresql://reservas_app@/canchas_db"

psql: error: connection to server failed:
FATAL:  permission denied for database
        "canchas_db"
DETAIL:  User does not have CONNECT privilege.
```

No es una convención de equipo: es un **REVOKE**  
**CONNECT** en  
infra/postgres/init/01-bases-y-roles.sql.

## Ocho reglas, y dónde vive cada una

|       | Regla                                      | Dónde vive                                           |
|-------|--------------------------------------------|------------------------------------------------------|
| RN-01 | Cancha, fecha y bloque de una hora         | <code>BookingService.crear · TimeSlot</code>         |
| RN-02 | Sin solapamiento, ni bajo concurrencia     | Servicio <code>y EXCLUDE</code> en la base           |
| RN-03 | Cada quien cancela lo suyo; el admin, todo | <code>BookingService.cancelar</code>                 |
| RN-04 | No se cancela una reserva ya empezada      | <code>Booking.sePuedeCancelarEn</code>               |
| RN-05 | Cancelar libera el bloque, sin borrar      | <code>Booking.cancelar</code>                        |
| RN-06 | Tope de reservas activas (3 por defecto)   | <code>BookingService.verificarTope</code>            |
| RN-07 | Solo el admin gestiona el catálogo         | <code>CourtService</code> en <code>ms-canchas</code> |
| RN-08 | Confirmada / Cancelada / Finalizada        | <code>Booking.estadoVigente</code>                   |

Las ocho están cubiertas por **73 pruebas automatizadas** en 12 clases. La correspondencia completa, en el informe (E1, §7).



## RN-02: validar en el servicio no basta

### La carrera

Dos peticiones sobre el mismo bloque consultan antes de que ninguna haya escrito. Ambas lo ven libre. Ambas insertan.

### Quién impone la regla

PostgreSQL. Si la carrera ocurre, la segunda inserción falla y el servicio la traduce a **409 Conflict**.

ConcurrenciaReservaIT lanza las dos a la vez, 10 veces seguidas, y exige que gane exactamente una.

```
ALTER TABLE reserva
ADD CONSTRAINT reserva_sin_solape
EXCLUDE USING gist (
    cancha_id WITH =,
    tsrange(fecha + hora_inicio,
            fecha + hora_fin, '[') WITH &&
) WHERE (estado = 'CONFIRMADA');
```

WHERE estado = 'CONFIRMADA' es lo que hace que cancelar libere el bloque (RN-05) sin borrar la fila, conservando la traza de RN-08 y el dato del reporte de cancelaciones.

# Los cuatro indicadores, sobre el rango que elija el administrador

1. Reservas por cancha y por deporte.
2. Ocupación por cancha: horas reservadas sobre horas disponibles.
3. Cancelaciones del período.
4. Canchas de mayor y menor demanda.

Solo lectura, solo administrador. Sin almacén analítico: `ms-reportes` compone por REST lo que ya está en `ms-reservas` y `ms-canchas`.

En la demo se crea una reserva y se vuelve aquí: [el número se mueve](#).



# Un comando, once contenedores, ningún requisito previo

1

`docker compose up`

11

contenedores

73

pruebas automatizadas

0

cosas que instalar

| Atributo       | Mecanismo concreto                                                                          |
|----------------|---------------------------------------------------------------------------------------------|
| Mantenibilidad | Dominio aislado de HTTP y de JPA; cada regla con su prueba                                  |
| Escalabilidad  | Servicios sin estado tras el gateway; se replican por separado                              |
| Seguridad      | Bases aisladas por credenciales; el navegador nunca alcanza un servicio                     |
| Disponibilidad | <code>healthcheck</code> por contenedor; un remote caído degrada una ruta, no la aplicación |

Ni Java, ni Node, ni Maven, ni PostgreSQL en la máquina: todo se compila dentro de los contenedores. Manual de despliegue aparte (E5).

## Como socio

1. Entrar y ver la disponibilidad de una cancha para hoy.
2. Reservar un bloque libre.
3. Intentar reservar **el mismo bloque** desde otra sesión: el sistema lo rechaza (**RN-02**).
4. Cancelar la reserva desde «mis reservas».
5. El bloque vuelve a estar libre (**RN-05**).

## Como administrador

6. Crear una cancha y verla en el catálogo del socio.
7. Cancelar la reserva de otro usuario (**RN-03**).
8. Abrir reportes: el número refleja lo que acaba de pasar.

### Si sobra tiempo

Reconstruir `mf-reportes` con el sistema en pie, para enseñar el despliegue independiente.

# Los seis criterios de aceptación

| Criterio del alcance (§7)                                    | Evidencia                                                          |
|--------------------------------------------------------------|--------------------------------------------------------------------|
| ✓ CA-1 El socio consulta, reserva y cancela                  | Demo, pasos 1–5;<br><code>BookingServiceCreacionTest</code>        |
| ✓ CA-2 El admin gestiona canchas y cancela cualquier reserva | Demo, pasos 6–7; <code>CourtServiceTest</code>                     |
| ✓ CA-3 No se reserva un bloque ocupado                       | Demo, paso 3; <b>RN-02</b> y<br><code>ConcurrenciaReservaIT</code> |
| ✓ CA-4 Reportes de ocupación y reservas por período          | Demo, paso 8; los cuatro indicadores                               |
| ✓ CA-5 Shell y $\geq 2$ remotes con Module Federation        | Tres remotes, desplegados por separado                             |
| ✓ CA-6 $\geq 2$ microservicios sobre PostgreSQL              | Cuatro servicios, tres bases aisladas                              |

**Lo que nos llevamos:** una regla crítica que solo vive en el código de aplicación no es una regla, es una intención; y la independencia de datos hay que hacerla imposible de violar.

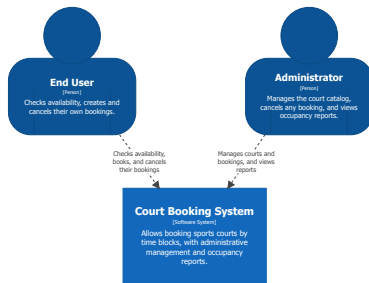
Gracias

¿Preguntas?

# Índice de respaldo

- Las cuatro vistas C4: contexto, contenedores, componentes de `ms-reservas`, despliegue
- Modelo entidad-relación
- Componentes del `shell`, `api-gateway` y del resto de servicios
- Reglas de negocio traducidas a respuestas HTTP
- Estrategia de pruebas
- Lista de comprobación previa a la demo
- Preguntas que esperamos

# Contexto (V-01)



System Context View: Court Booking System

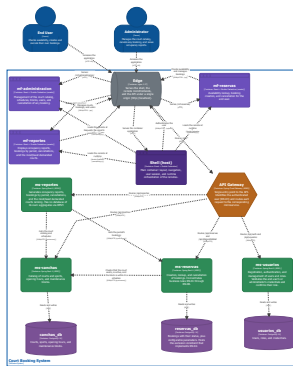
Level 1 — The system and its users.

Saturday, August 29, 2026 at 2:07 AM Ecuador Time

Socio y administrador frente al sistema. Sin integraciones externas: es parte del alcance acordado.



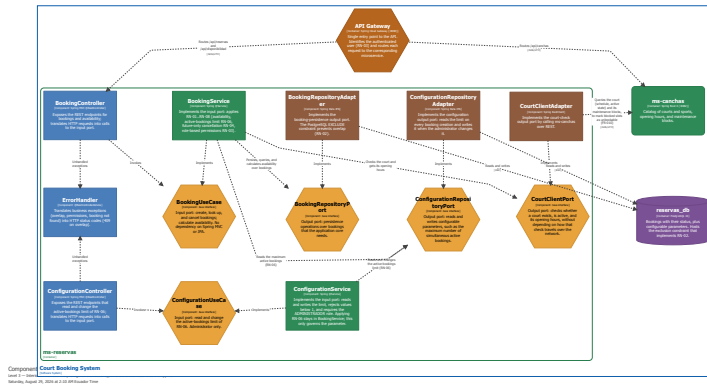
# Contenedores (V-02)



Container-based Cloud Billing System  
Source: [https://www.ibm.com/cloud/cloud-architecture](#)  
Modified: August 20, 2020 by IBM Cloud Security Team

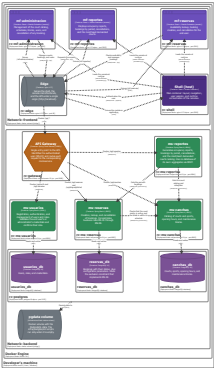
Las once piezas. El **edge** es el único conectado a las dos redes de Docker: es lo que impide alcanzar un microservicio saltándose el gateway.

# Componentes de ms-reservas (V-03)



Donde viven las reglas de negocio. Adaptadores fuera, dominio dentro: el dominio no conoce ni HTTP ni JPA.

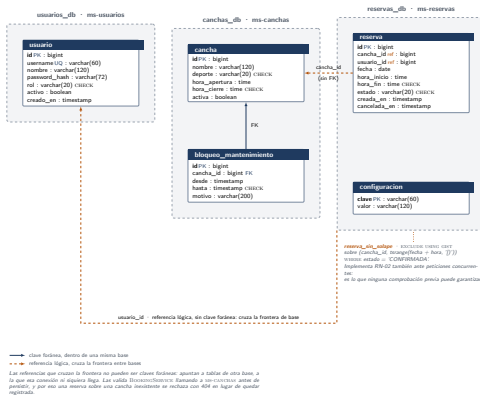
## Despliegue (V-04)



**Deployment View Court Booking System - Local/Extranet/Corporate**  
 Illustration of the system in a desktop environment.  
 Copyright © 2008, All Rights Reserved.

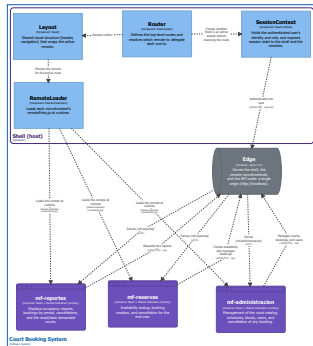
Dos redes Docker, once contenedores y un volumen. Solo el `edge` toca las dos redes.

# Modelo entidad-relación



Tres bases separadas, no tres esquemas del mismo modelo. Lo que cruza la frontera es una referencia por identificador, no una clave foránea.

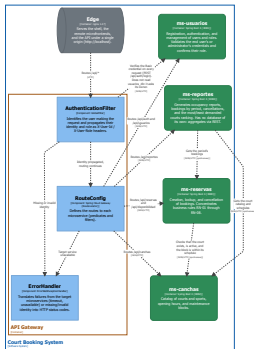
# Componentes del shell



Component View: Court Booking System - Shell (host)  
Level 1 -> Structure of the shell (host), session, and reservation loading  
Saturday, August 26, 2023 at 10:00 AM (UTC-05:00)

Cómo el host resuelve los remotes y comparte la sesión.

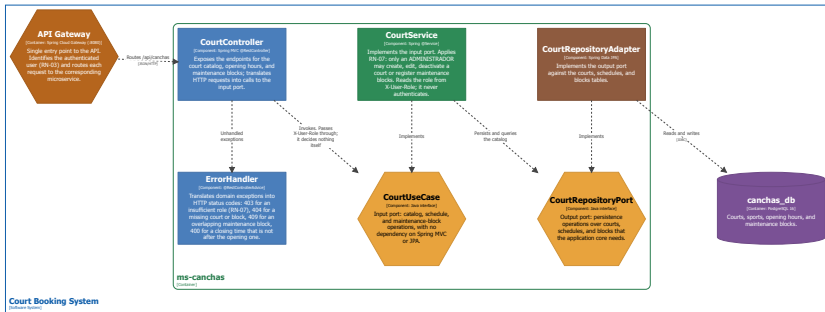
# Componentes del api-gateway



Component View: Court Booking System - API Gateway  
Version 1.0 - Version of the API Gateway component and its dependencies  
Saturday, August 25, 2023 at 2:07 PM Ecuador Time

Dónde se verifica la credencial antes de rutear.

# Componentes de ms-canchas

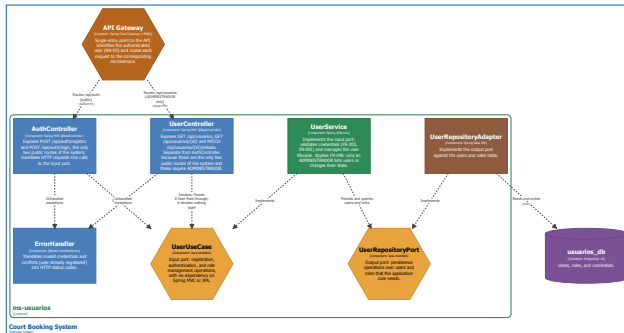


Component View: Court Booking System - ms-canchas

Level 3 — Interior of ms-canchas (hexagonal architecture): catalog, schedules, and blocks.

Saturday, August 29, 2026 at 2:07 AM Ecuador Time

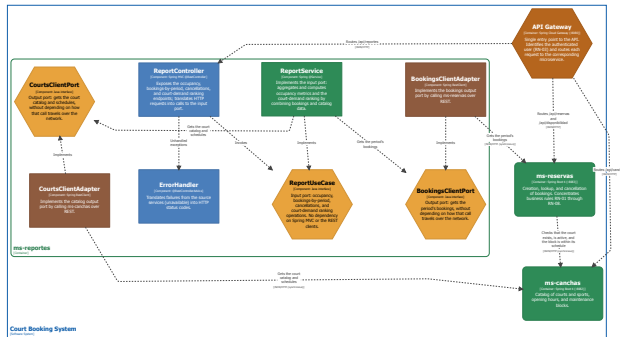
# Componentes de ms-usuarios



Component View: Court Booking System - ms-usuarios  
Level 3 -> Interior of ms-usuarios (package architecture): registration and basic role-based authentication.  
Saturday, August 26, 2023 at 2:47 AM Ecuador Time



# Componentes de ms-reportes



Component View: Court Booking System - ms-reportes  
Level 3 -> Interior of ms-reportes (hierarchical architecture) R007 aggregation, no database of its own.  
Saturday, August 24, 2024 at 2:07 PM Ecuador Time

El único servicio sin base de datos: compone por HTTP.

## Reglas de negocio → respuestas HTTP

| Situación                        | Código | Regla / motivo                                                          |
|----------------------------------|--------|-------------------------------------------------------------------------|
| Bloque ya ocupado                | 409    | <b>RN-02</b> — igual si lo detecta el servicio o la restricción EXCLUDE |
| Tope de reservas alcanzado       | 409    | <b>RN-06</b>                                                            |
| Cancelar una reserva ya iniciada | 409    | <b>RN-04</b>                                                            |
| Cancelar una reserva ajena       | 403    | <b>RN-03</b> — y no 404: fingir que no existe ocultaría el motivo       |
| Reserva o cancha inexistente     | 404    |                                                                         |
| Cancha inactiva o bloqueada      | 422    | Existe, pero no admite esa reserva                                      |
| Cuerpo inválido                  | 400    | Validación del <i>request</i>                                           |
| Sin credencial válida            | 401    | Rechazado en el gateway                                                 |
| ms-canchas no responde           | 503    | El servicio del que se depende está caído                               |

Fuente: ms-reservas/.../adapter/in/web/ErrorHandler.java.

- 73 métodos de prueba en 12 clases.
- Unitarias de dominio y aplicación sobre las reglas: BookingServiceCreacionTest, BookingServiceCancelacionTest, CourtServiceTest, ReportServiceTest, UserServiceTest.
- De integración con PostgreSQL real: ConcurrencyReservaIT, que es la única forma de probar RN-02 bajo concurrencia.
- Del gateway: AuthenticationFilterTest, sobre la propagación de identidad y rol.

## Por qué ConcurrencyReservaIT repite

Una carrera que se gana una vez puede ser suerte. La prueba lanza las dos peticiones simultáneas **diez veces seguidas** y exige que en las diez gane exactamente una.

## Lista de comprobación previa a la demo

- `docker compose up -d --wait` lanzado **antes** de entrar al aula; `docker compose ps` con los once servicios en `healthy`.
- Dos navegadores (o uno normal y uno privado) con sesión iniciada: uno como socio, otro como administrador.
- Datos recién sembrados: `docker compose down -v && docker compose up -d --wait`, para que los reportes no arrastren reservas viejas.
- Zoom del navegador al 125 %: lo que se lee en el portátil no se lee en el proyector.
- Notificaciones del sistema operativo silenciadas.
- **Plan B:** un vídeo corto de la demo, ya abierto en el escritorio.

## Preguntas que esperamos

- ¿Por qué microservicios para un sistema de este tamaño?
- ¿Qué pasa si `ms-canchas` está caído cuando alguien reserva? (*Devuelve 503: no se crea una reserva sobre una cancha que no se pudo validar.*)
- ¿Por qué no una cola de mensajes entre servicios?
- ¿Module Federation no acopla los remotes al shell por la versión de React? (*Sí: por eso React va como `singleton` declarado en los cuatro.*)
- ¿Cómo evitáis que un remote roto tumbe toda la aplicación?
- ¿Cuánto de esto lo escribió el modelo y cuánto vosotros?